

Functions & Views Avançado

Scalar UDFs, Inline TVFs, Multi-Statement TVFs, Views Padrão, Updatable Views e Indexed e Indexed Views no SQL Server.

fn_exemplo.sql

T-SQL

```
-- Exemplo: Inline Table-Valued Function (iTVF)
CREATE FUNCTION
    @Cidade VARCHAR(50)
RETURNS TABLE
AS
RETURN
    SELECT
        FROM
        WHERE @Cidade
```

✓ iTVF · macro expansion

Ln 7 · Col 1

2

TÓPICOS

24

SLIDES

~60min

DURAÇÃO

Agenda do Módulo

Dois tópicos cobrindo funções escalares/de tabela e views padrão/atualizáveis/indexadas.

DURAÇÃO ~60MIN

☰ TÓPICO 3.1

01 / 02

Funções Escalares e de Tabela

Scalar UDFs e o problema RBAR, Scalar UDF Inlining (SQL 2019+), iTVFs como macros, macros, mstVFs e estimativas cegas de cardinalidade.

1. 01Teoria: Scalar UDF, RBAR, Inlining (SQL 2019+)
2. 02Teoria: iTVF vs mstVF — quando usar cada uma
3. 03Cenário real: desconto em 100k pedidos (RBAR vs set-based)
4. 04Código: ANTES (Scalar) vs DEPOIS (iTVF) + tabela comparativa
5. 05Erros comuns, anti-patterns e desafio

🕒 ~30 MIN

📊 TÓPICO 3.2

02 / 02

Implementação, Atualização e Restrições de Views

Views Padrão (SCHEMABINDING + CHECK OPTION), Updatable Views (INSTEAD OF triggers), Indexed Views (materialização e custo oculto).

1. 01Teoria: Views Padrão + SCHEMABINDING + CHECK OPTION
2. 02Teoria: Updatable Views + INSTEAD OF triggers
3. 03Teoria + checklist: Indexed Views (requisitos estritos)
4. 04Cenário real: dashboard de agregação diária lenta
5. 05Código: SCHEMABINDING, Indexed View, INSTEAD OF + desafio

🕒 ~30 MIN

Total estimado: ~60 minutos · 24 slides · Exemplos práticos em T-SQL

TÓPICO 01 / 02

01

Funções Escalares e de Tabela

Da sintaxe básica ao comportamento interno: RBAR (Row-By-Agonizing-Row), Scalar UDF Inlining (SQL 2019+), iTVFs como macros e mstVFs com estimativas cegas de cardinalidade.

- 01** → Scalar UDF: sintaxe, RBAR e impacto de performance
- 02** → Scalar UDF Inlining (SQL 2019+): pré-condições e limitações
- 03** → iTVF (Inline TVF): macro expansion no plano principal
- 04** → mstVF (Multi-Statement TVF): estimativas cegas de cardinalidade
- 05** → Cenário real + código ANTES/DEPOIS + anti-patterns + desafio

Scalar UDF e o problema RBAR

O QUE É SCALAR UDF

Uma Scalar User-Defined Function (UDF) retorna um ÚNICO valor escalar (INT, VARCHAR, DATETIME, DATETIME, etc.). É definida com `CREATE FUNCTION`, retorna um tipo escalar via `RETURNS`, e o corpo fica dentro de `BEGIN...END`. Pode ser usada em SELECT, WHERE, computed columns constraints CHECK.

O PROBLEMA: RBAR

RBAR (Row-By-Agonizing-Row) é o termo cunhado por Jeff Moden: o SQL Server executa a Scalar UDF uma vez POR LINHA do result set. Para 100k linhas, são 100k invocações da função. Cada invocação tem custo de contexto (context switch) — o otimizador não consegue inlineá-la (pré-2019), tratando-a como black box.

IMPACTO MEDIDO

- 100x** mais lenta que lógica equivalente em JOIN/CROSS APPLY
- ~1ms** por invocação (mesmo para função trivial) — overhead de contexto
- 0 reusos** de plano — cada invocação é tratada independentemente

fn_CalcularDesconto.sql

SCALAR UDF

```
-- Scalar UDF: calcula desconto progressivo
CREATE FUNCTION
    @Valor DECIMAL 10 2
RETURNS DECIMAL 10 2
AS
BEGIN
    DECLARE @Desconto DECIMAL 5 2    0
    IF @Valor > 1000
        SET @Desconto = 15.0
    ELSE IF @Valor > 500
        SET @Desconto = 10.0
    ELSE
        SET @Desconto = 5.0
    RETURN @Valor * @Desconto / 100
END
GO

-- Uso problemático (chamada por linha):
SELECT
FROM
    -- 100k linhas = 100k invocações
```

⚠ 100k invocações = ~1-8s

RBAR ativo

SCALAR UDF INLINING

Scalar UDF Inlining (SQL 2019+)

O recurso do projeto Froid que automaticamente transforma Scalar UDFs em código inline no plano principal.

01

SQL SERVER ≤ 2017

Scalar UDFs sempre sofrem RBAR. Context Context switch linha-a-linha. Plano mostra mostra operador de função isolado.

LEGADO · RBAR

02

PESQUISA FROID (2018)

Paper da Microsoft Research demonstra que 60% das UDFs poderiam ser automaticamente inlineadas. Base do projeto Froid.

PESQUISA

03

SQL SERVER 2019+

Froid integrado ao SQL Server 2019 (database compatibility level 150). UDFs elegíveis são automaticamente inlineadas.

INLINE AUTOMÁTICO

04

INLINE NO PLANO PRINCIPAL

Código da UDF é expandido no SELECT. Otimizador enxerga tudo como uma única query — gera plano ótimo, reusa cache.

PERFORMANCE IGUAL A TVFS

Pré-condições para inlining

- Função não pode ter acesso a tabelas (sem SELECT/INSERT/UPDATE/DELETE dentro).
- Não pode ter múltiplas atribuições (multi-assignment).
- Não pode chamar outras UDFs não-inlineáveis.
- Database compatibility level ≥ 150 (SQL 2019+).
- Habilitar via T-SQL: ALTER DATABASE SCOPED CONFIGURATION SET TSQL_SCALAR_UDF_INLINING = ON.

Limitações (ainda não inlinea)

- UDFs com acesso a tabelas (mstVF ou com SELECT interno).
- UDFs com chamadas recursivas.
- UDFs com funções não-determinísticas em certos contextos.
- UDFs com construções proceduralmente complexas (GOTO, loops com multi-branch).
- Mesmo com inlining, UDFs com lógica muito complexa podem não ser inlineadas — verifique com SET STATISTICS XML.

◆ Verificar se inlining ocorreu: SET STATISTICS XML ON — procure <UDFInline>

iTVF vs mstVF — a diferença que importa

Duas categorias de Table-Valued Functions com comportamento interno completamente diferente.



INLINE TABLE-VALUED

iTVF (Inline Table-Valued Function)

TRATADA COMO MACRO

COMO FUNCIONA

O SQL Server trata a iTVF como uma macro: expande o corpo da função diretamente no plano diretamente no plano da consulta principal. O otimizador enxerga tudo como uma única query — gera plano ótimo, reusa cache, permite paralelismo.

SINTAXE

```
CREATE FUNCTION dbo.fn_ClientesAtivosPorCidade
(@Cidade VARCHAR(50))
RETURNS TABLE
AS
RETURN (
SELECT Id, Nome, Email, Cidade
FROM dbo.Clientes
WHERE Cidade = @Cidade AND Ativo = 1
);
```

PERFORMANCE

- ✓ Performance equivalente a subquery ou view
- ✓ Plano de execução ótimo (com índices, JOINS)
- ✓ Estimativa de cardinalidade correta



MULTI-STATEMENT TVF

mstVF (Multi-Statement TVF)

BLACK BOX PARA O OTIMIZADOR

COMO FUNCIONA

A mstVF declara uma tabela de retorno (@Result) e popula via INSERTs dentro de BEGIN...END. O SQL Server NÃO consegue expandir o corpo no plano principal — trata a função como black box, retornando a tabela sem estatísticas.

SINTAXE

```
CREATE FUNCTION dbo.fn_TopClientesMST
(@Quantidade INT)
RETURNS @Result TABLE (ClienteId INT, Total DECIMAL(10,2))
AS
BEGIN
INSERT INTO @Result
SELECT TOP (@Quantidade) ClienteId, SUM(Total)
FROM dbo.Vendas GROUP BY ClienteId;
RETURN;
END
```

PERFORMANCE

- ⚠ Estimativa de cardinalidade cega (assume 1 linha!)
- ⚠ Plano ruim — otimizador trata como black box
- ⚠ Sem reuso de índices das tabelas base

✓ Regra de ouro: SEMPRE que possível, use iTVF em vez de Scalar UDF ou mstVF

 COMPARATIVO SINTÉTICO

Scalar UDF vs iTVF vs mstVF vs View

Síntese para consulta rápida — quando usar cada um, quando evitar.

CARACTERÍSTICA	SCALAR UDF	iTVF	mstVF	VIEW PADRÃO
Retorno	1 valor escalar	Tabela (1 SELECT)	Tabela (corpo procedural)	Tabela (SELECT salvo)
Contexto execução	badRBR (linha-a-linha)	okMacro (expande no plano)	badBlack box	okSubquery (expande no plano)
Plano execução	badIsolado por linha	okInline no plano principal	badIsolado da consulta	okInline no plano principal
Estimativa cardinalidade	warnN/A	okCorreta	badCEGA (assume 1 linha)	okCorreta
Performance (100k linhas)	bad~ 8s	ok~ 50ms	warn~ 2-5s	ok~ 50ms
Inlining possível	warnSim (SQL 2019+, com limites)	okSempre (nativa)	badNão	okSempre (nativa)
Quando usar	Formatação, computed columns	Consultas reutilizáveis	Raramente — prefira SP	Abstração, segurança
Quando evitar	badConsultas em volume	okN/A (sempre boa)	badSempre que possível	badAninhar views

◆ Print esta tabela para os alunos — referencial para troubleshooting de performance

CENÁRIO REAL

Desconto em 100k Pedidos — o calcanhar de Aquiles

Um caso clássico de Scalar UDF em loop de produção.

O PROBLEMA EM PRODUÇÃO

Um e-commerce tem uma SP de fechamento de pedidos que, para cada pedido, calcula o desconto aplicável. A desconto aplicável. A lógica foi implementada em uma Scalar UDF `dbo.fn_CalcularDesconto` — chamada na SELECT principal. Em média, 100k pedidos são processados por dia em batch noturno. minutos para completar — deveria levar 30 segundos.

SINTOMAS OBSERVADOS

- →SP com 100k linhas em Pedidos, chamando `fn_CalcularDesconto` por linha.
- →CPU time muito maior que elapsed time (context switches).
- →Plano de execução mostra operador <UDF> isolado — não inline.
- →SET STATISTICS IO mostra 100k logical reads na tabela Pedidos + custo invisível da UDF.
- →DBA tenta adicionar índice em Pedidos — não ajuda (gargalo está na UDF).
- →Inspeccionar `sys.dm_exec_requests`: sessão com high CPU usage, baixo IO.

Diagnóstico Correto

NÃO É ÍNDICE · É RBAR

A Scalar UDF `fn_CalcularDesconto` sofre RBAR: é invocada 100k vezes, cada uma com custo de ~1ms de context switch. O otimizador não consegue ver dentro da UDF — não consegue otimizar. Adicionar índices não ajuda porque a UDF já está isolando a lógica.

COMO CONFIRMAR

- →SET STATISTICS TIME ON — comparar CPU time vs elapsed time.
- →SET STATISTICS XML ON — procurar elemento <UDF> no plano XML.
- →`sys.dm_exec_function_stats` — ver `execution_count` e `total_elapsed_time` da UDF.
- →Habilitar Scalar UDF Inlining (SQL 2019+) — se aplicável.

PRÓXIMOS PASSOS

Veremos no slide 9 o código problemático (ANTES) e no slide 10 a versão a versão refactorada como iTVF (DEPOIS).

⚠ Sintoma clássico: CPU >> IO — UDF esconde o custo real

ANTES - PROBLEMATICO

Scalar UDF com RBAR

Funcao escalar invocada por linha - 100k invocacoes para 100k pedidos.

O QUE ESTA ERRADO

- ▶ Scalar UDF invocada uma vez por linha da SELECT.
- ▶ Optimizador trata UDF como black box - nao consegue otimizar.
- ▶ Cada invocacao = context switch (~1ms por chamada).
- ▶ 100k linhas = ~100k context switches = ~8s de CPU.
- ▶ Plano de execucao mostra operador <Table Scan> + <UDF> isolado.
- ▶ SET STATISTICS IO: 100k logical reads + custo invisivel da UDF.

fn_CalcularDesconto.sql

⚠ ANTES - RBAR

```
-- =====
-- SP: usp_FechamentoPedidos (VERSAO ANTES - PROBLEMATICA)
-- Sintoma: 8 minutos para 100k pedidos - deveria ser 30s
-- =====
CREATE OR ALTER PROCEDURE dbo.usp_FechamentoPedidos
    @DataFechamento DATE
AS
BEGIN
    SET NOCOUNT ON
    -- Scalar UDF invocada por linha:
    SELECT
        PedidoId
        Total
        dbo.fn_CalcularDescontoTotal AS Desconto -- 100k invocacoes
        Total - dbo.fn_CalcularDescontoTotal AS Liquido -- + 100k
    FROM dbo.Pedidos
    WHERE DataPedido = @DataFechamento
END
GO
-- Scalar UDF que causa o problema:
CREATE FUNCTION dbo.fn_CalcularDesconto
    @Valor DECIMAL(10, 2)
RETURNS DECIMAL(10, 2)
AS
BEGIN
    DECLARE @Pct DECIMAL(5, 2) = 0
    IF @Valor > 1000 SET @Pct = 15.0
    ELSE IF @Valor > 500 SET @Pct = 10.0
    ELSE SET @Pct = 5.0
    RETURN @Valor * @Pct / 100
END
GO
-- Resultado (SET STATISTICS TIME ON):
-- CPU time = 4723 ms, elapsed time = 8123 ms
-- Logical reads = 2453 (na tabela Pedidos) + custo invisivel da UDF
```

⚠ 200k invocacoes da UDF (100k x 2 colunas)

~8s - deveria ser 30ms

⚠ Mesmo SQL Server 2019+ nao inlineia esta UDF (tem multi-assignment IF)

DEPOIS - OTIMIZADO

iTVF + CROSS APPLY

Logica extraida para Inline TVF - otimizador enxerga enxerga tudo como uma query.

O QUE MUDOU

- ✓ Logica extraida da Scalar UDF para uma iTVF (Inline Table-Valued Function).
- ✓ iTVF e tratada como macro - expande no plano principal.
- ✓ CROSS APPLY invoca a iTVF por linha, mas sem context switch.
- ✓ Otimizador gera plano unico para toda a consulta.
- ✓ Estimativa de cardinalidade correta (usa estatisticas da tabela).
- ✓ Resultado: ~50ms em vez de ~8s - speedup de ~160x.

fn_CalcularDesconto_iTVF.sql

✓ DEPOIS - iTVF

```
-- =====
-- SP: usp_FechamentoPedidos (VERSAO DEPOIS - iTVF)
-- Speedup ~160x: 8s -> 50ms
-- =====
-- 1. iTVF que retorna valor calculado + valor liquido
CREATE OR ALTER FUNCTION dbo.fn_CalcularDescontoTVF
    @Valor DECIMAL(10, 2)
RETURNS TABLE
AS
RETURN
    SELECT
        @Valor AS ValorOriginal
    CASE
        WHEN @Valor = 1000 THEN @Valor * 0.15
        WHEN @Valor = 500 THEN @Valor * 0.10
        ELSE @Valor * 0.05
    END AS Desconto
    @Valor CASE
        WHEN @Valor = 1000 THEN @Valor * 0.15
        WHEN @Valor = 500 THEN @Valor * 0.10
        ELSE @Valor * 0.05
    END AS Liquido
GO

-- 2. SP usa CROSS APPLY para invocar a iTVF
CREATE OR ALTER PROCEDURE dbo.usp_FechamentoPedidos
    @DataFechamento DATE
AS
BEGIN
    SET NOCOUNT ON
    SELECT
        PedidoId
        Total
        Desconto
        Liquido
    FROM dbo.Pedidos
    CROSS APPLY dbo.fn_CalcularDescontoTVF(@Total) AS Desconto
    WHERE DataPedido = @DataFechamento
END
GO

-- Resultado (SET STATISTICS TIME ON):
-- CPU time = 47 ms, elapsed time = 52 ms
-- Logical reads = 2453 (mesmo) - sem custo invisivel
```

✓ iTVF e a solucao padrao para logica calculada em volume

 ANTI-PATTERNS

Erros Comuns e Anti-patterns

O que os desenvolvedores costumam errar ao criar Functions.



PROBLEMA

Scalar UDF em SELECT com milhares de linhas

Dev usa Scalar UDF em SELECT com 1M+ linhas - RBAR ativo, RBAR ativo, 100k+ invocações, performance desastre.

CORREÇÃO

Migrar para iTVF com CROSS APPLY. Ou, se SQL 2019+, 2019+, garantir compatibility level >= 150 e verificar inlining via SET STATISTICS XML.



PROBLEMA

mstVF em JOINS grandes

mstVF em JOIN com tabela grande - estimativa cega (assume 1 linha), plano ruim, table scan.

CORREÇÃO

Refactor para iTVF (se lógica simples) ou para SP (se complexa). Evitar mstVF em consultas em volume.



PROBLEMA

Funções não-determinísticas em computed columns

Criar computed column com GETDATE() ou NEWID() - muda a cada SELECT, não pode ser indexada.

CORREÇÃO

Usar funções determinísticas em computed columns. Para datas variáveis, usar coluna regular populada por trigger ou SP.



PROBLEMA

UDF com efeitos colaterais (acesso a tabelas)

Scalar UDF com SELECT interno - não pode ser inlineada, inlineada, RBAR perpetuo, dificulta troubleshooting.

CORREÇÃO

NUNCA colocar SELECT/INSERT/UPDATE/DELETE dentro de Scalar UDF. Para lógica com acesso a dados, usar SP.



PROBLEMA

UDFs aninhadas (chamadas em cadeia)

fn_A chama fn_B que chama fn_C - cada uma com seu custo. Em 100k linhas, é multiplicador de RBAR.

CORREÇÃO

Achatamento: implementar toda a lógica em uma única iTVF. Ou usar CROSS APPLY para compor resultados sem context switch.



PROBLEMA

UDFs reutilizadas sem baseline de performance

UDF criada há anos, usada em várias SPs - ninguém mediu performance. Virou legado intocável.

CORREÇÃO

Monitorar sys.dm_exec_function_stats mensalmente. Para UDFs com alto total_elapsed_time, avaliar migração para iTVF.

✓ Padrão ouro: SEMPRE preferir iTVF sobre Scalar UDF em volume

◆ DESAFIO - TOPICO 3.1

Desafio: Migrar Scalar UDF para iTVF

Aplique tudo o que voce aprendeu sobre RBAR e iTVF.

ENUNCIADO

Voce e DBA de uma administradora de cartoes. A SP `dbo.usp_CalcularCashback` calcula, para cada transacao transacao do mes, o cashback aplicavel baseado em: (a) categoria do estabelecimento (1%, 2% ou 5%); (b) VIP (+ 0.5%); (c) valor da transacao. A SP invoca a Scalar UDF `dbo.fn_CalcularCashback` por linha. Para transacoes/mes, a SP leva 12 minutos. Tarefas:

TAREFAS

- 01 Diagnosticar: confirme RBAR usando SET STATISTICS TIME/IO e sys.dm_exec_function_stats (cite os valores os valores esperados).
- 02 Refactor: criar uma iTVF equivalente que calcule tudo em conjunto (categoria + VIP + valor).
- 03 Reescrever a SP usando CROSS APPLY para invocar a iTVF.
- 04 Comparar performance: CPU time, logical reads, elapsed time - antes vs depois.

DICAS

- ▶ A iTVF deve retornar uma tabela com: TransacaoId, CashbackCategoria, CashbackVIP, CashbackTotal.
- ▶ Use CASE WHEN para substituir os IFs da Scalar UDF.
- ▶ CROSS APPLY na SP: FROM dbo.Transacoes t CROSS APPLY dbo.fn_CalcularCashbackTVF(t.Valor, t.CategoriaId, t.ClienteVip) c.
- ▶ Verifique se o plano XML mostra a iTVF como inline (procure <SqlInlineFunction> = 1).

Critérios de Avaliacao

CHECKLIST - 7 ITENS

- check_box_outline_blank 01 - Diagnostico correto com correto com SET STATISTICS + DMV
- check_box_outline_blank 02 - iTVF criada com RETURNS TABLE e RETURN (SELECT ...)
- check_box_outline_blank 03 - CASE WHEN para substituir IFs
- check_box_outline_blank 04 - CROSS APPLY na SP
- check_box_outline_blank 05 - Plano XML mostra iTVF como inline
- check_box_outline_blank 06 - Speedup esperado: ~12 min -> ~5s
- check_box_outline_blank 07 - Documentacao das mudancas

TOPICO 02 / 02

02

Implementacao, Atualizacao e Restricoes de Views

Views Padrao (SCHEMABINDING + CHECK OPTION), Updatable Views (INSTEAD OF triggers), Indexed triggers), Indexed Views (materializacao e custo oculto de escrita).

- 01 → Views Padrao: SCHEMABINDING + CHECK OPTION
- 02 → Updatable Views + INSTEAD OF triggers
- 03 → Indexed Views (Materialized): requisitos e custo oculto
- 04 → Cenario real: dashboard de agregacao diaria lenta
- 05 → Codigo: SCHEMABINDING, Indexed View, INSTEAD OF + desafio

 VIEWS PADRAO

Views Padrao: SCHEMABINDING + CHECK OPTION

Abstracao e seguranca com garantias de consistencia.



OPCAO 01

View Padrao

ABSTRACAO E SEGURANCA

SINTAXE

```
CREATE VIEW dbo.vw_ClientesAtivos
AS
SELECT ClienteId, Nome, Email
FROM dbo.Clientes
WHERE Ativo = 1;
```

QUANDO USAR

Abstrair complexidade (JOINS, formulas), limitar acesso a colunas (seguranca), padronizar consultas repetidas.

EXEMPLO

Usuarios recebem SELECT apenas na view - nao ganham ganham SELECT direto na tabela Clientes (oculta colunas colunas sensiveis como CPF, RG).



OPCAO 02

WITH SCHEMABINDING

TRAVA O SCHEMA DAS TABELAS BASE

SINTAXE

```
CREATE VIEW dbo.vw_ClientesAtivos
WITH SCHEMABINDING
AS
SELECT ClienteId, Nome, Email
FROM dbo.Clientes;
-- tabelas base nao podem ser ALTER/DROP
-- sem antes DROP VIEW
```

QUANDO USAR

Quando a view e critica (usada por SPs, reports). Garante que ALTER TABLE nas tabelas base nao quebre a view.

EXEMPLO

ALTER TABLE dbo.Clientes DROP COLUMN CPF falha com falha com erro: "Cannot DROP COLUMN CPF because it is because it is referenced by object vw_ClientesAtivos". vw_ClientesAtivos".



OPCAO 03

WITH CHECK OPTION

GARANTE CONSISTENCIA EM MODIFICACOES

SINTAXE

```
CREATE VIEW dbo.vw_ClientesAtivos
AS
SELECT ClienteId, Nome, Ativo
FROM dbo.Clientes
WHERE Ativo = 1
WITH CHECK OPTION;
-- INSERT/UPDATE com Ativo=0 e rejeitado
```

QUANDO USAR

Em views de filtro (ex: WHERE Ativo = 1). Impede INSERT/UPDATE que criaria linha que nao seria visivel pela view.

EXEMPLO

INSERT INTO vw_ClientesAtivos (Nome, Ativo) VALUES VALUES ('Joao', 0) falha com erro: "The attempted insert or insert or update failed because the target view either either specifies WITH CHECK OPTION..."

◆ Boa pratica: SCHEMABINDING em views de producao; CHECK OPTION em views de filtro

 UPDATABLE VIEWS

Updatable Views + INSTEAD OF Triggers

Quando e como executar INSERT/UPDATE/DELETE através de uma view.

REGRAS DE ATUALIZACAO

Uma view é atualizável quando atende a TODOS os critérios abaixo. Se falhar em qualquer um, qualquer um, INSERT/UPDATE/DELETE não são possíveis - a menos que use INSTEAD OF triggers.

✓ **Uma tabela base so:** view sem JOIN, UNION, INTERSECT.

✓ **Sem agregacoes:** sem GROUP BY, HAVING, DISTINCT.

✓ **Sem colunas calculadas:** colunas derivadas (Preço * Qt) não podem ser UPDATE.

✓ **Sem TOP/OFFSET:** não limita linhas.

✓ **Todas as colunas NOT NULL presentes:** view deve incluir todas colunas obrigatórias.

✓ **Sem funcoes nao-deterministicas:** sem GETDATE(), NEWID() em colunas.

Views com JOIN de 2+ tabelas em um-para-um a partir da chave primaria ainda podem ser atualizáveis - com restrições.

Workaround: INSTEAD OF Triggers

CONTORNA RESTRICOES COMPLEXAS

INSTEAD OF triggers são disparados NO LUGAR da operação original. Permitem views atualizáveis mesmo com JOINS, agregações, ou colunas calculadas - o trigger recebe os dados e decide o que fazer.

CASOS DE USO

- ► View com JOIN entre 2+ tabelas (quer inserir em ambas).
- ► View com coluna calculada (precisa persistir valor calculado na base).
- ► View sobre tabela em outro schema (precisa de validação extra).
- ► View de auditoria (precisa logar modificação antes de aplicar).

EXEMPLO CONCEPTUAL

```
CREATE TRIGGER tr_vw_Pedidos_Instead
ON dbo.vw_PedidosCompleto
INSTEAD OF INSERT
AS
BEGIN
-- Recebe INSERTED e decide em quais
-- tabelas base (Clientes, Pedidos) inserir
INSERT INTO dbo.Clientes ...
INSERT INTO dbo.Pedidos ...
END
```

Detalhes no slide 21 - código completo.

⚠ View com JOIN de N:N ou agregação JAMais é atualizável sem INSTEAD OF

 INDEXED VIEWS

Indexed Views (Materialized Views)

Transformando uma view em estrutura física - acelera leituras pesadas com custo de escrita.

O QUE SÃO INDEXED VIEWS

Uma Indexed View (também chamada Materialized View) é uma view cujo result set é persistido fisicamente no disco. Ao criar um UNIQUE CLUSTERED INDEX na view, o SQL Server materializa o resultado - as linhas são armazenadas como uma tabela real. Consultas subsequentes que usam a view podem ler diretamente do índice clusterizado, sem re-executar a SELECT original.

COMO FUNCIONA

- 01 Criar view com WITH SCHEMABINDING (obrigatório).
- 02 Criar UNIQUE CLUSTERED INDEX na view (materialização).
- 03 SQL Server mantém o índice sincronizado com as tabelas base.
- 04 Consultas que usam a view podem usar o índice - ou o otimizador pode usar automaticamente automaticamente (matching).

CUSTO OCULTO DE ESCRITA

Cada DML (INSERT/UPDATE/DELETE) nas tabelas base que afeta o result set da view dispara atualização do índice clusterizado. Em ambientes OLTP com alto volume de escrita, esse overhead pode degradar performance de INSERT/UPDATE em 20-50%.

Trade-off: Pros vs Contras

USE CASES IDEAL X PESSIMO

PROS - IDEAL PARA

- ✓ Agregações pesadas (SUM, COUNT, AVG) em grandes volumes.
- ✓ JOINS custosos repetidos frequentemente (ex: dimensões de data).
- ✓ Dados históricos (pouca escrita, muita leitura).
- ✓ Consultas que combinam múltiplas tabelas lentas.

CONTRAS - PESSIMO PARA

- ⚠ Overhead de escrita: cada DML nas tabelas base dispara update da view.
- ⚠ Requisitos estritos: SCHEMABINDING, deterministic functions, sem OUTER JOIN, sem UNION (ver slide 17).
- ⚠ Custo de storage: a view ocupa espaço em disco como uma tabela.
- ⚠ Lock contention em ambientes OLTP de alto volume.

Regra geral: bom para OLAP / DW; ruim para OLTP de alto volume.

◆ Verifique requisitos detalhados no slide 17 (checklist)

✓ CHECKLIST DE PRODUÇÃO

Requisitos estritos para Indexed View

10 condições que DEVEM estar satisfeitas antes de `CREATE UNIQUE CLUSTERED INDEX`.



CONFIGURAÇÃO E SCHEMA

5 REQUISITOS

- ✓SET ANSI_NULLS ON e SET QUOTED_IDENTIFIER ON —na criação da view e das view e das tabelas base.
- ✓WITH SCHEMABINDING —obrigatório na criação da view.
- ✓Schema-bound: todas as tabelas base referenciadas também devem ter SCHEMABINDING em suas dependências.
- ✓Nomes de schema em duas partes (dbo.vw_NomeView) —não usar nomes sem schema.
- ✓Database compatibility level e configuração de ANSI corretos.



FUNÇÕES E CONSTRUÇÃO

5 REQUISITOS

- ✓Apenas funções determinísticas: DATEADD, CONVERT (estilos), CASE. Proibido: GETDATE(), NEWID(), RAND().
- ✓Sem OUTER JOIN (LEFT, RIGHT, FULL). Apenas INNER JOIN.
- ✓Sem UNION / UNION ALL / INTERSECT / EXCEPT.
- ✓Sem subqueries (não pode ter SELECT aninhado na lista de colunas).
- ✓Sem TOP / ORDER BY / OFFSET na definição da view.

⚠ **NOTA:** Edições do SQL Server: somente Enterprise Edition faz matching automático (usa a Indexed View mesmo em queries que não referenciam a view). Standard Edition e Express exigem hint `WITH (NOEXPAND)` explícita para usar a Indexed View.

✓ Print este checklist — 10 requisitos fundamentais

CENÁRIO REAL

Dashboard de Vendas: 30s para abrir

Um caso clássico de agregação pesada sem Indexed View.

O PROBLEMA EM PRODUÇÃO

Um ERP de gestão tem um dashboard de vendas que agrega por dia, categoria e região. Para o mês corrente (média de 5M linhas na tabela Vendas), o dashboard demora 30 segundos para abrir. A view `dbo.vw_VendasDashboard` faz GROUP BY em 3 dimensões com JOIN em 4 tabelas (Vendas, Produtos, Categorias, Regiões).

SINTOMAS OBSERVADOS

- Dashboard abre em 30s para o mês corrente —deveria ser <500ms.
- `SET STATISTICS TIME ON`: CPU time = 28s, elapsed time = 30s (CPU-bound).
- Plano de execução: Hash Match (agregação) + Clustered Index Scan em Vendas (5M linhas).
- DBA tenta criar índice em Vendas(DataVenda) — não ajuda (ainda precisa agregar).
- DBA tenta criar SP em vez de view — mesma performance (problema não é o nome do objeto).
- Dashboard é consultado a cada 5 min por ~50 usuários concorrentes — 30s × 50 = lock contention.

Diagnóstico Correto

VIEW AGREGADA SEM MATERIALIZAÇÃO

A view `vw_VendasDashboard` re-executa a agregação a cada chamada. 5M linhas × GROUP BY × 4 JOINS = custo de ~28s de solução é materializar o resultado em uma Indexed View — as agregações passam a ser pré-computadas.

COMO CONFIRMAR

- `SET STATISTICS TIME ON` — confirma CPU-bound (CPU = elapsed).
- `sys.dm_exec_query_stats` — exec_count da query = 50/hora (alta frequência).
- Plano XML mostra <Hash Match> com alto estimated rows (5M).
- Avaliar se a view atende aos 10 requisitos do slide 17.

IMPACTO ESPERADO

Com Indexed View: dashboard em ~200ms (speedup de 150x). Trade-off: cada INSERT em Vendas dispara update da view (~5ms overhead). Em volume de 10k vendas/dia, overhead total = 50s/dia — compensa.

⚠ Sintoma: view de agregação + alta frequência de leitura + baixa frequência de escrita

◆ CÓDIGO PRÁTICO

View com SCHEMABINDING + CHECK OPTION

Protege o schema das tabelas base e garante consistência em modificações via view.

PONTOS-CHAVE

- **WITH SCHEMABINDING** impede ALTER TABLE/DROP nas tabelas base.
- **WITH CHECK OPTION** rejeita INSERT/UPDATE INSERT/UPDATE que cria linhas invisíveis à view.
- Schema-bound: nomes em 2 partes (dbo.Tabela) obrigatório.
- SET ANSI_NULLS ON e SET QUOTED_IDENTIFIER ON requeridos para criação.
- GRANT SELECT na view, não na tabela base (segurança + Ownership Chaining).
- Para executar: rodar todo o batch (VIEW + GRANT + testes).

vw_ClientesAtivos.sql

SCHEMABINDING + CHECK OPTION

```
-- =====
-- VIEW: vw_ClientesAtivos
-- Protege Schema das tabelas base (SCHEMABINDING)
-- Garante consistência em modificações (CHECK OPTION)
-- =====
SET ANSI_NULLS ON
SET QUOTED_IDENTIFIER ON
GO

CREATE OR ALTER VIEW
WITH SCHEMABINDING
AS
    SELECT

        FROM          1
        WHERE
        AND            '2020-01-01'
        WITH CHECK OPTION
    GO

-- Teste 1: tentativa de ALTER TABLE falha
-- (com SCHEMABINDING, não é possível dropar coluna referenciada)
ALTER TABLE        DROP COLUMN
-- ERRO: Cannot DROP COLUMN Email because it is
-- referenced by object vw_ClientesAtivos

-- Teste 2: tentativa de INSERT inconsistente falha
-- (com CHECK OPTION, não pode inserir linha que não seria visível)
INSERT INTO

VALUES
    1001 'Joao''joao@x.com'SP'2026-09-01' 0
-- ERRO: The attempted insert failed because target view
-- specifies WITH CHECK OPTION

-- Teste 3: INSERT consistente funciona
INSERT INTO

VALUES
    1002 'Maria''maria@x.com'RJ'2026-09-01' 1
-- OK: 1 row affected

-- Conceder SELECT apenas na view (segurança)
GRANT SELECT ON      TO
-- app_user não recebe SELECT direto em dbo.Clientes
```

✓ Schema travado · consistência garantida

SCHEMABINDING + CHECK OPTION

✓ Padrão defensivo: schema protegido + dados consistentes

 INDEXED VIEW

Criando uma Indexed View

Materialização de agregação para acelerar acelerar dashboard de vendas.

3 PASSOS

- 01** Criar VIEW com **WITH SCHEMABINDING** (obrigatório).
- 02** Criar **UNIQUE CLUSTERED INDEX** na view (materialização).
- 03** Consultas que usam a view são aceleradas automaticamente.

IMPACTO MEDIDO

30s → 200ms	Dashboard de vendas (150x speedup)
+5ms	por INSERT em Vendas (overhead de escrita)
~50MB	de storage adicional para o para o índice

 vw_VendasDashboard_Indexed.sql

```
-- =====
-- INDEXED VIEW: Materialização de dashboard de vendas
-- Speedup: 30s -> 200ms para dashboard de vendas
-- =====
SET ANSI_NULLS ON
SET QUOTED_IDENTIFIER ON
GO

-- 1. VIEW com SCHEMABINDING (obrigatório para Indexed View)
CREATE OR ALTER VIEW
WITH SCHEMABINDING
AS
SELECT

        COUNT_BIG      AS
        SUM             AS
        SUM             AS

FROM
INNER JOIN            ON
INNER JOIN            ON
GROUP BY

GO

-- 2. UNIQUE CLUSTERED INDEX materializa a view
CREATE UNIQUE CLUSTERED INDEX
ON
GO

-- 3. Consulta que se beneficia (matching automático em Enterprise)
SELECT

FROM
WHERE
ORDER BY
        DESC
-- Antes: 30s (Hash Match + Clustered Index Scan em 5M linhas)
-- Depois: 200ms (Clustered Index Seek na view materializada)

-- 4. Hint NOEXPAND (necessário em Standard/Express Edition)
SELECT
FROM
WHERE
        WITH
        '2026-09-01'

-- 5. Manutenção: REBUILD do índice da view (como qualquer índice)
ALTER INDEX
ON
REBUILD
GO
```

✓ View materializada · plano ótimo automático

30s → 200ms (150x)

✓ NOEXPAND hint necessário em Standard Edition para forçar uso da Indexed View

INSTEAD OF TRIGGER

View com JOIN atualizável via INSTEAD OF

Contorna a restrição de views com JOIN para para INSERT/UPDATE/DELETE.

PONTOS-CHAVE

- View com JOIN normalmente **não é atualizável**.
- **INSTEAD OF INSERT** substitui o INSERT original — decide em quais tabelas inserir.
- **INSERTED** é uma pseudo-tabela com os os dados recebidos.
- Permite validações e splits antes de tocar nas tabelas base.
- Só um INSTEAD OF por operação (INSERT, UPDATE, DELETE) por view.

vw_PedidoCompleto_Instead.sql

INSTEAD OF INSERT

```
-- =====
-- VIEW com JOIN: precisaria de INSTEAD OF para ser atualizável
-- =====

-- 1. View com JOIN (não é atualizável diretamente)
CREATE OR ALTER VIEW
AS
    SELECT
        AS
        AS

    FROM
    INNER JOIN        ON
GO

-- Tentativa de INSERT direto falha:
INSERT INTO
VALUES 'Joao'joao@x.c2026-09-12' 1500
-- ERRO: View or function has insert target columns
-- that span multiple base tables

-- 2. INSTEAD OF INSERT — contorna a restrição
CREATE OR ALTER TRIGGER
ON
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON

    -- INSERTED é a pseudo-tabela com os dados recebidos
    -- Para cada linha, verificar se cliente existe; se não, criar

    -- Insert em Clientes (se não existir)
    INSERT INTO
    SELECT
    FROM
    WHERE NOT EXISTS
    SELECT FROM
    WHERE

    -- Insert em Pedidos (usando ClienteId recém-criado ou existente)
    INSERT INTO
    SELECT

    FROM
    INNER JOIN        ON
END
GO

-- 3. Agora o INSERT pela view funciona
INSERT INTO
VALUES 'Joao'joao@x.c2026-09-12' 1500
```

✓ View com JOIN agora é atualizável

INSTEAD OF INSERT

✓ Considere usar SP em vez de INSTEAD OF se a lógica ficar muito complexa

 ANTI-PATTERNS · TÓPICO 3.2

Anti-patterns de Views em produção

Os 6 erros mais comuns que DBAs e devs cometem com Views.



PROBLEMA · 01

View sem SCHEMABINDING

Dev cria view sem `SCHEMABINDING` — qualquer `ALTER` `ALTER TABLE` nas tabelas base pode quebrar a view (referências a colunas removidas).

CORREÇÃO

SEMPRE usar `WITH SCHEMABINDING` em views de produção. Garante que `ALTER TABLE/DROP` nas tabelas base falhe até a view ser `DROP` primeiro.



PROBLEMA · 02

View aninhando view (perf cascata)

Dev cria `vw_A` que referencia `vw_B` que referencia `vw_C` `vw_C` — otimizador expande tudo, mas em alguns casos gera plano péssimo.

CORREÇÃO

Achatamento: `vw_A` deve referenciar diretamente as tabelas base. Se reusabilidade é necessária, usar SP com cross-database calls.



PROBLEMA · 03

Indexed View sem calcular ROI

Dev cria Indexed View para acelerar leituras sem avaliar custo de escrita — em OLTP de alto volume, overhead de `INSERT/UPDATE` degrada produção.

CORREÇÃO

Calcular ROI antes: medir (a) speedup de leitura (ganho) vs (b) overhead de escrita (custo). Se volume de escrita > 10% do volume de leitura, não usar Indexed View.



PROBLEMA · 04

SELECT * em view

Dev usa `SELECT *` na view — quando tabela base nova coluna, a view não se atualiza; quando coluna removida, a view quebra.

CORREÇÃO

SEMPRE listar colunas explícitas na view. `SELECT *` é anti-pattern em views — quebra compatibilidade com dependentes.



PROBLEMA · 05

View para abstrair JOIN em vez de SP

Dev cria view para abstrair JOIN complexo que só é usado usado por 1 SP — camada de abstração desnecessária, desnecessária, dificulta troubleshooting.

CORREÇÃO

Usar SP para abstrair lógica procedural. View deve ser usada para reusabilidade transversal (múltiplas SPs, múltiplas aplicações).



PROBLEMA · 06

CHECK OPTION esquecido em views de filtro

View de filtro (`WHERE Ativo = 1`) sem `CHECK OPTION` — dev consegue `INSERT` com `Ativo = 0`, criando linha invisível à própria view.

CORREÇÃO

SEMPRE usar `WITH CHECK OPTION` em views de filtro. Impede dados inconsistentes que violam a intenção da view.

✓ Padrão ouro: `SCHEMABINDING` sempre; `CHECK OPTION` em filtros; ROI em Indexed Views

◆ DESAFIO · TÓPICO 3.2

Otimizar dashboard de vendas

Cenário realista — você é o DBA responsável pela solução.

ENUNCIADO

Você é DBA de uma administradora. A view `dbo.vw_PedidosDashboard` agrega pedidos por região, categoria e mês. Em média, 3M linhas na tabela Pedidos. O dashboard usa essa view e demora **25 segundos** para abrir. O dashboard é consultado a cada 5 min por ~30 usuários. Volume de escrita: 5k pedidos/dia.

TAREFAS

- 01 Diagnosticar: confirmar se o problema é a view (`SET STATISTICS TIME/IO + sys.dm_exec_query_stats`).
- 02 Avaliar se a view atende aos 10 requisitos do slide 17 para virar Indexed View.
- 03 Se sim: criar a Indexed View (`CREATE UNIQUE CLUSTERED INDEX`). Se não: propor refactor da view para atender.
- 04 Calcular ROI: speedup de leitura vs overhead de escrita — justificar se a Indexed View vale a pena.

DICAS

- Confirme que todas as funções na view são determinísticas (sem `GETDATE`, `NEWID`).
- Use `INNER JOIN` apenas — sem `OUTER JOIN`.
- `COUNT_BIG(*)` em vez de `COUNT(*)` para grupos.
- Em Standard Edition, use `WITH (NOEXPAND)` para forçar uso da Indexed View.

Critérios de Avaliação

CHECKLIST · 8 ITENS

- ✓ 01— Diagnóstico correto com `SET STATISTICS + DMV`
- ✓ 02— Avaliação dos 10 requisitos (todos ou nenhum)
- ✓ 03— Identificação de requisitos não atendidos
- ✓ 04— Refactor da view se necessário
- ✓ 05— `CREATE UNIQUE CLUSTERED INDEX` na view
- ✓ 06— Cálculo de ROI (speedup leitura vs overhead escrita)
- ✓ 07— Hint `NOEXPAND` (em Standard Edition)
- ✓ 08— Documentação das mudanças

SÍNTESE DO MÓDULO

O que vimos hoje

01

TÓPICO 3.1

Funções Escalares e de Tabela

Scalar UDFs e o problema RBAR (context switch linha-a-linha), Scalar UDF Inlining (SQL 2019+, projeto Froid), iTVFs como macros (expande no plano principal), mstVFs como black boxes (estimativas cegas), tabela comparativa de performance (Scalar 8s vs iTVF 50ms = 160x speedup) e anti-patterns.

02

TÓPICO 3.2

Implementação, Atualização e Restrições de Views

Views Padrão (SCHEMABINDING + CHECK OPTION para segurança e consistência), Updatable Views (regras estritas + INSTEAD OF triggers como workaround), Indexed Views/Materialized Views (10 requisitos estritos, acelera agregações pesadas com custo de escrita), cenário real de dashboard de vendas (30s → 200ms) e anti-patterns.

→ Próximos Passos

- Reproduzir todos os códigos no SSMS local (SQL Server Developer Edition).
- Resolver o desafio do slide 12 (migrar Scalar UDF para iTVF).
- Resolver o desafio do slide 23 (otimizar dashboard com Indexed View).
- Habilitar Scalar UDF Inlining em databases legadas (compat level 150).
- Auditar views de produção: verificar SCHEMABINDING + CHECK OPTION.

   proxima_aula.sql

SQL

```
-- Próxima aula: Módulo 4 — Triggers
-- DML, DDL e LOGON
SELECT TOP 5
FROM
ORDER BY      DESC
```

Módulo 4: Triggers — DML, DDL, LOGON e padrões de uso responsável.